MEGA APUNTES DE DISENO DE BASES DE DATOS

Teoria para el examen + Ejercicios practicos resueltos

Documento de estudio. Cubre: modelo relacional, modelo Entidad-Relacion (E-R) y extendido (MERE), paso a tablas, restricciones de integridad, dependencias funcionales, axiomas de Armstrong, cierre de atributos, claves, normalizacion (1FN, 2FN, 3FN, BCNF) y desnormalizacion. Al final hay una bateria de ejercicios tipo examen con soluciones.

BLOQUE 1. EL MODELO RELACIONAL

1.1. Que es el modelo relacional

El modelo relacional representa TODA la informacion mediante una unica estructura logica: la tabla (formalmente, relacion). No importa la complejidad del problema: el resultado final siempre es un conjunto de tablas.

- **Fila o tupla:** cada registro o instancia individual.
- **Columna o atributo:** cada propiedad de los registros.
- **Valor atomico:** cada celda contiene un unico valor indivisible (exigencia de la 1FN). No se permiten listas ni estructuras anidadas.

1.2. Por que se dice que es uniforme

Porque usa una sola estructura (la tabla) tanto para los datos como para las relaciones entre ellos:

- Las entidades (Clientes, Productos) se convierten en tablas.
- Las relaciones se representan con tablas (N:M) o con columnas adicionales / claves foraneas (1:N).
- Los atributos multivaluados se extraen a su propia tabla.
- Los vinculos se basan en VALORES (coincidencia entre clave primaria y clave foranea), no en punteros fisicos.

Ventajas de la uniformidad: sencillez (una sola estructura que aprender), flexibilidad (varias vistas externas sobre la misma base) e independencia.

1.3. Independencia de los datos

- **Independencia fisica:** el modo de almacenamiento en disco (indices, archivos, hardware, compresion) NO afecta a la definicion logica de las tablas. Permite cambiar de disco, reorganizar archivos o crear indices sin tocar el esquema.
- **Independencia logica:** se puede modificar el esquema logico (anadir atributos, dividir tablas al normalizar) SIN afectar a las aplicaciones que usan los datos. Las vistas ayudan a que la aplicacion siga "viendo" lo mismo aunque la estructura cambie.

Regla rapida para no confundirlas:
- Fisica = protege la definicion de las tablas frente a cambios en el almacenamiento.
- Logica = protege a las aplicaciones frente a cambios en la estructura de las tablas.

1.4. Fases del diseno de una base de datos

1. **Diseno conceptual:** entender los datos del minimundo. Resultado: modelo E-R / MERE (independiente del SGBD).
2. **Diseno logico:** esquema de tablas, columnas, tipos y relaciones (independiente del producto concreto).

3. Diseno fisico: implementacion real en disco (indices, ficheros, hardware).

BLOQUE 2. MODELO ENTIDAD-RELACION (E-R)

2.1. Elementos basicos

- **Entidad:** objeto del mundo real con existencia propia (rectangulo en Chen).
- **Atributo:** propiedad de una entidad (ovalo en Chen).
- **Relacion / vinculo:** asociacion entre entidades (rombo en Chen).

2.2. Tipos de atributos

- **Simple / atomico:** un solo valor.
- **Compuesto:** se descompone (Direccion = calle + numero + CP).
- **Multivaluado:** admite varios valores (Telefonos). Se representa con ovalo doble. Al pasar a tablas SIEMPRE genera una tabla aparte.
- **Derivado / calculado:** se obtiene a partir de otros (Edad a partir de FechaNacimiento).
- **Identificador / clave:** atributo subrayado que identifica la entidad.

2.3. Razon de cardinalidad (maximos)

Numero MAXIMO de instancias de una entidad que se pueden asociar con una instancia de otra.

- **1:1 (uno a uno):** un DEPARTAMENTO tiene un DIRECTOR y viceversa.
- **1:N (uno a muchos):** un CURSO tiene muchos ALUMNOS, pero cada alumno pertenece a un solo curso.
- **N:M (muchos a muchos):** un EMPLEADO trabaja en varios PROYECTOS y un proyecto tiene varios empleados.

2.4. Restriccion de participacion (minimos)

- **Total (obligatoria):** la entidad debe participar siempre en la relacion (minimo 1). Se dibuja con linea doble. En tablas, la FK se define NOT NULL.
- **Parcial (opcional):** la entidad puede existir sin la relacion (minimo 0). Linea simple. La FK admite NULL.

CLAVE DE EXAMEN: cardinalidad = maximo (con cuantos como mucho); participacion = minimo (es obligatorio o no).

2.5. Entidad fuerte vs entidad debil

Caracteristica	Entidad fuerte	Entidad debil
Existencia	Propia e independiente	Depende de una entidad propietaria
Clave primaria	Propia y completa	Compuesta: FK del dueno + clave parcial
Notacion Chen	Rectangulo simple	Rectangulo doble
Participacion	Parcial o total	Siempre TOTAL respecto a su dueno

- **Atributo discriminante (clave parcial):** atributo que distingue instancias de la entidad debil que pertenecen al MISMO dueno. No es unico en toda la tabla, solo localmente.
- Ejemplo: EMPLEADO (PK: Nss) y FAMILIAR (discriminante: NombreFamiliar). La tabla FAMILIAR tiene PK = (Nss, NombreFamiliar).
- Si la entidad debil depende de DOS entidades fuertes, su PK combina las claves de ambas + su

discriminante, y el vinculo identificador deja de ser binario (puede ser ternario).
- Si el dueno desaparece, la entidad debil debe eliminarse (ON DELETE CASCADE): es una anomalia de borrado controlada.

BLOQUE 3. MODELO E-R EXTENDIDO (MERE)

3.1. Jerarquias: especializacion y generalizacion

Una superclase (entidad generica) se divide en subclases (entidades especificas). Las subclases HEREDAN todos los atributos y relaciones de la superclase y anaden los suyos propios.

Las jerarquias se controlan con dos restricciones independientes:

A) Restriccion de disyuncion:
- **Disjunta (d):** una instancia pertenece como maximo a UNA subclase (Empleado = Tecnico O Administrativo, nunca ambos).
- **Solapada (s / o):** una instancia puede pertenecer a VARIAS subclases a la vez (una Persona puede ser Alumno y Profesor simultaneamente).

B) Restriccion de completitud:
- **Total:** toda instancia de la superclase DEBE pertenecer a alguna subclase (linea doble).
- **Parcial:** una instancia puede no pertenecer a ninguna subclase (linea simple).

Se combinan: total+disjunta, total+solapada, parcial+disjunta, parcial+solapada.

3.2. Agregacion

REGLA CLAVE DE EXAMEN: la agregacion es estrictamente necesaria cuando hay que modelar una relacion que a su vez participa en otro vinculo con una tercera entidad.

- Problema que resuelve: en el E-R original una relacion (rombo) NO puede conectarse con otra relacion. Solo se conectan entidades.
- Solucion: se "abstrae" la relacion y sus entidades participantes tratandolas como una unica entidad compuesta (metadefinicion). Asi se puede crear un nuevo vinculo con una tercera entidad.
- Representacion en Chen: un rectangulo envolvente que rodea la relacion y sus entidades.
- Ejemplo: la relacion TRABAJA_EN entre EMPLEADO y PROYECTO. Si un SUPERVISOR evalua esa participacion concreta, se agrega TRABAJA_EN y se conecta con SUPERVISOR.

CUIDADO: no confundir agregacion con "jerarquia solapada y total". Esas son restricciones de pertenencia de una especializacion, NO una tecnica de abstraccion de relaciones. Siempre que leas "una relacion participa en otro vinculo", la respuesta es AGREGACION.

3.3. Arco o relacion exclusiva

Indica exclusividad mutua: una instancia puede relacionarse con UNA de varias opciones, pero no con varias a la vez. Ejemplo: un PAGO lo realiza un CLIENTE_PARTICULAR o una EMPRESA, pero no ambos.

Se dibuja con una linea curva (arco) que cruza las relaciones en conflicto. En SQL no tiene traduccion automatica: se implementa con varias FK + restriccion CHECK (solo una FK con valor, el resto NULL) o con una columna discriminadora.

3.4. Relacion reflexiva (recursiva o unaria)

Una entidad se relaciona consigo misma; los registros juegan roles distintos. Ejemplo: EMPLEADO supervisa EMPLEADO.

- Implementacion (1:N): se anade una FK en la propia tabla que apunta a su PK (id_supervisor ->

id_empleado).
- La FK suele ser NULL: el cargo mas alto (CEO) no tiene supervisor. Si fuera NOT NULL se crearia un bucle imposible.
- Usos: jerarquias organizativas, categorias anidadas, listas de materiales.
- Se recomienda nombrar los roles (supervisor / supervisado).

BLOQUE 4. NOTACIONES DE DIAGRAMAS

4.1. Chen

Usa rectangulos (entidades), ovalos (atributos) y rombos (relaciones). La cardinalidad se anota con 1, N, M sobre las lineas. Entidad debil = rectangulo doble; relacion identificadora = rombo doble; discriminante = ovalo con linea discontinua subrayada.

4.2. Barker (entornos Oracle)

Muy orientada a reglas de negocio.
- **Linea continua:** relacion obligatoria (must be).
- **Linea discontinua:** relacion opcional (may be).
- **Barra de identificacion (UID bar) "|":** la relacion forma parte del identificador unico (PK) de la entidad. Sustituye al rectangulo doble de Chen. En tablas, esa FK pasa a formar parte de la PK. Ejemplo: LINEA_PEDIDO se identifica con ID_Pedido (heredado) + Numero_Linea.
- **Rombo (no-transferibilidad):** una vez establecida la relacion, NO puede cambiarse a otra instancia. Ejemplo: un CAPITULO no puede moverse a otro LIBRO.

Diferencia clave: UID bar define QUIEN ERES (identidad); el rombo define que la relacion es PERMANENTE (no cambia de dueno).

4.3. Pata de Gallo (Crow's Foot)

Simbolos visuales: una barra para "uno" y una pata de tres lineas para "muchos"; circulo para opcional.

BLOQUE 5. PASO DEL MODELO E-R AL RELACIONAL

5.1. Entidades

- Entidad fuerte -> tabla con su propia PK.
- Entidad debil -> tabla con PK compuesta = FK del dueno + discriminante.

5.2. Atributo multivaluado

REGLA DE ORO: un atributo multivaluado SIEMPRE genera una tabla aparte (para cumplir la 1FN / atomicidad).
1. Se elimina el atributo de la tabla original.
2. Se crea una tabla nueva.
3. La nueva tabla incluye la PK de la entidad como FK; su PK es (PK_entidad + valor del atributo).

Ejemplo: Cliente(DNI, Nombre) con Telefono multivaluado -> Telefonos(DNI, Telefono) con PK compuesta.

5.3. Relacion 1:1

Se propaga la PK de un lado como FK en el otro (con restriccion UNIQUE). Si ambos lados son de

participacion parcial y pocos registros se relacionan, conviene una tabla separada para evitar exceso de NULL. Se coloca la FK en el lado de participacion mas cercana a total para minimizar nulos.

5.4. Relacion 1:N

Se propaga la PK del lado "1" como FK en la tabla del lado "N". NO se crea tabla nueva.
- Participacion total en el lado N -> FK NOT NULL.
- Participacion parcial -> FK admite NULL.
- No se hace al reves porque el lado "1" tendria que guardar una lista de valores (violaria la 1FN).

5.5. Relacion N:M

SIEMPRE se crea una tabla intermedia (de union/asociativa):
- Columnas: las PK de las dos entidades como FK.
- PK de la tabla intermedia = union de ambas FK (clave compuesta).
- Los atributos propios de la relacion (Fecha, Horas...) se colocan en esta tabla.

Ejemplo: Trabaja_En(Nss, NumProyecto, Horas).

BLOQUE 6. RESTRICCIONES DE INTEGRIDAD

6.1. Restricciones en el modelo relacional (SQL)

- **NOT NULL:** prohibe valores nulos en una columna.
- **UNIQUE:** prohibe duplicados en una columna o conjunto.
- **PRIMARY KEY:** identifica de forma unica cada fila; es NOT NULL + UNIQUE.
- **FOREIGN KEY (referencial):** un valor en la tabla hija debe existir en la tabla padre; evita registros huerfanos.
- **CHECK:** condicion logica sobre los valores permitidos (p. ej. salario > 1000).
- **DEFAULT:** valor por defecto si no se especifica.

6.2. Integridad referencial y acciones en cascada

Garantiza que cada valor de una FK referencia una PK existente. Controla insercion, actualizacion y borrado. Estrategias ON DELETE / ON UPDATE:

- **RESTRICT (por defecto):** bloquea la operacion si hay hijos dependientes.
- **CASCADE:** al borrar/actualizar el padre, se propaga a los hijos. Tipico en entidades debiles.
- **SET NULL:** pone la FK de los hijos a NULL (solo si la participacion es parcial).

6.3. Restricciones estructurales del MER

- Razon de cardinalidad (1:1, 1:N, N:M).
- Restriccion de participacion (total / parcial).

6.4. Restricciones en jerarquias (MERE)

- Disyuncion (disjunta / solapada).
- Completitud (total / parcial).

6.5. Otros tipos

- **Dependencias funcionales:** restriccion que proviene de la semantica de los datos; medio formal para saber si un diseno es correcto.
- **Restricciones informativas:** el optimizador SQL las usa para rendimiento, pero el gestor no las verifica.

- **Integridad referencial en cascada:** acciones automaticas CASCADE / SET NULL / RESTRICT.

BLOQUE 7. DEPENDENCIAS FUNCIONALES (DF)

7.1. Definicion

Una dependencia funcional X -> Y es una restriccion unidireccional entre dos conjuntos de atributos: X determina funcionalmente a Y si, siempre que dos filas coinciden en X, OBLIGATORIAMENTE coinciden en Y.

- Para cada valor de X existe uno y solo un valor de Y.
- X = determinante (el que manda); Y = dependiente (el que sigue).
- Es UNIDIRECCIONAL: X -> Y no implica Y -> X. (El DNI determina el Nombre, pero el Nombre no determina el DNI.)
- Si encuentras dos filas con igual X y distinto Y, la DF no se cumple: estado ilegal/inconsistente.

7.2. Para que sirven

- Identificar claves: una clave candidata determina funcionalmente a TODOS los atributos.
- Detectar redundancia y evitar anomalias de insercion, actualizacion y borrado.
- Base de la normalizacion.

7.3. Tipos de DF

- **Trivial:** X -> Y donde Y esta contenido en X (p. ej. AB -> A). Siempre se cumple, no aporta informacion.
- **No trivial:** Y no esta contenido en X.
- **Parcial:** un atributo no primo depende de SOLO UNA PARTE de una clave compuesta. (Problema de 2FN.)
- **Transitiva:** un atributo no primo depende de otro atributo no primo (X -> Y, Y -> Z, luego X -> Z). (Problema de 3FN.)

7.4. Anomalias que evita una buena DF

- **Insercion:** no poder anadir datos por faltar otros obligatorios.
- **Actualizacion:** cambiar un dato en una fila y olvidarlo en sus repeticiones -> inconsistencia.
- **Borrado:** perder informacion util al eliminar un registro.

El "sindrome de la hoja de calculo" es una tabla gigante con redundancia y sin validacion; la normalizacion lo cura.

BLOQUE 8. AXIOMAS DE ARMSTRONG Y REGLAS DERIVADAS

8.1. Axiomas basicos (1974)

1. **Reflexividad:** si Y esta contenido en X, entonces X -> Y. (Genera las DF triviales.) Por eso el cierre empieza con X+ = X (un atributo se determina a si mismo).
2. **Aumento:** si X -> Y, entonces XZ -> YZ para cualquier Z. (Si conozco X conozco Y; si ademas conozco Z, sigo conociendo todo.)
3. **Transitividad:** si X -> Y e Y -> Z, entonces X -> Z. (Eslabon intermedio que conecta los extremos.)

8.2. Reglas derivadas

- **Union:** si X -> Y y X -> Z, entonces X -> YZ. (Mismo determinante; se fusionan los dependientes.)
- **Descomposicion:** si X -> YZ, entonces X -> Y y X -> Z. (Es la inversa de la union.)
- **Pseudotransitividad:** si X -> Y y WY -> Z, entonces WX -> Z.

Estas reglas permiten deducir nuevas DF, simplificar esquemas (dependencias reducidas) y calcular el cierre.

BLOQUE 9. CIERRE DE ATRIBUTOS (X+)

9.1. Que es

El cierre X+ es el conjunto de TODOS los atributos que se pueden determinar funcionalmente a partir de X bajo un conjunto de DF. Es la "lista de todo lo que puedes conocer si conoces X".

9.2. Algoritmo paso a paso

1. Inicializar: X+ = X (por reflexividad, X se determina a si mismo).
2. Expandir: recorre las DF. Si el lado izquierdo de una DF ya esta contenido en X+, anade su lado derecho a X+.
3. Repetir: vuelve a recorrer las DF; al crecer X+ puedes desbloquear nuevas DF.
4. Parar: cuando una pasada completa no anade nada nuevo.

9.3. La "prueba de la clave"

- Si X+ contiene TODOS los atributos de la relacion -> X es SUPERCLAVE.
- Si ademas X es minimo (ningun subconjunto propio de X determina todo) -> X es CLAVE CANDIDATA.
- Si X+ NO contiene todos los atributos -> X NO es clave (solo determina los atributos de su cierre: dependencias reducidas).

BLOQUE 10. CLAVES Y TIPOS DE ATRIBUTOS

10.1. Tipos de clave

- **Superclave:** conjunto de atributos que identifica de forma unica cada fila (puede tener atributos sobrantes).
- **Clave candidata:** superclave MINIMA (sin atributos sobrantes).
- **Clave primaria:** la clave candidata elegida como identificador oficial; es NOT NULL y unica.
- **Clave alternativa:** las claves candidatas no elegidas como primaria.
- **Clave foranea (FK):** atributo que referencia la PK de otra (o la misma) tabla.

Ejemplo: si {DNI} identifica a una persona, entonces {DNI, Color_Ojos} es superclave pero NO candidata (le sobra Color_Ojos). {DNI} es la clave candidata.

10.2. Atributo primo vs no primo

- **Primo:** forma parte de ALGUNA clave candidata.
- **No primo:** no forma parte de ninguna clave candidata (atributo puramente descriptivo).

REGLA DE ORO: si TODOS los atributos de una tabla son primos, la tabla esta automaticamente en 3FN (las reglas de 2FN y 3FN solo restringen a los atributos NO primos).

BLOQUE 11. NORMALIZACION

Proceso escalonado que estructura las tablas para eliminar redundancia y anomalias. Cada forma normal exige cumplir la anterior.

11.1. Primera Forma Normal (1FN)

- Se aplica a TODA tabla (clave simple o compuesta).
- Exige atomicidad: cada celda un unico valor indivisible; sin listas ni grupos repetidos.
- Los atributos multivaluados se extraen a una tabla aparte.

11.2. Segunda Forma Normal (2FN)

- Esta en 1FN y ningun atributo NO primo depende PARCIALMENTE de la clave (debe depender de la clave COMPLETA).
- Solo puede violarse con clave compuesta. Con clave simple, la 2FN es automatica (no hay "partes" de la clave).
- Solucion: separar en una tabla el trozo de clave que causa la dependencia parcial junto con el atributo dependiente.

Ejemplo: REGALA(Codigo_Regalo, DNI) con Nombre_Regalo. Nombre_Regalo depende solo de Codigo_Regalo -> dependencia parcial. Se separa REGALO(Codigo_Regalo, Nombre_Regalo).

11.3. Tercera Forma Normal (3FN)

- Esta en 2FN y ningun atributo NO primo depende TRANSITIVAMENTE de la clave (ningun atributo no primo depende de otro atributo no primo).
- Relevante tambien con clave simple.
- Atajo: si todos los atributos son primos, esta automaticamente en 3FN.

Ejemplo: Pedido(ID_Pedido, Fecha, ID_Cliente, Nombre_Cliente) con ID_Pedido -> ID_Cliente y ID_Cliente -> Nombre_Cliente. Nombre_Cliente depende transitivamente de ID_Pedido a traves de ID_Cliente. Se separa Cliente(ID_Cliente, Nombre_Cliente) y Pedido(ID_Pedido, Fecha, ID_Cliente FK).

Resolucion general de una transitiva A -> B -> C:
1. Detectar A (clave) -> B (no primo) -> C (no primo).
2. Crear tabla nueva con B como PK y mover C alli.
3. En la tabla original, B queda como FK.

11.4. Forma Normal de Boyce-Codd (BCNF, "3.5FN")

- Version mas estricta de la 3FN: TODO determinante de una DF no trivial debe ser SUPERCLAVE.
- Diferencia con 3FN: la 3FN permite X -> Y si Y es atributo primo aunque X no sea superclave; la BCNF elimina esa excepcion.
- Aparece tipicamente con claves candidatas compuestas y solapadas.
- Verificacion: para cada DF X -> Y, calcula X+. Si X+ no contiene todos los atributos, X no es superclave y se viola la BCNF.

11.5. Resumen de aplicacion

Forma	Clave simple	Clave compuesta	Requisito
1FN	Si	Si	Valores atomicos
2FN	Automatica	Si	Sin dependencias parciales

3FN	Si	Si	Sin dependencias transitivas
BCNF	Si	Si	Todo determinante es superclave

BLOQUE 12. DESNORMALIZACION

Estrategia de OPTIMIZACION que anade redundancia DELIBERADA a un esquema ya normalizado para acelerar la lectura. No es un mal diseno: la fuente de verdad siguen siendo las tablas normalizadas.

12.1. Cuando aplicarla

Cuando el rendimiento de lectura es critico y NO se resuelve con indices ni cache: cargas con muchas lecturas, uniones en abanico (3-5 tablas), agregaciones constantes, SLAs en milisegundos.

12.2. Tecnicas

- **Columnas redundantes:** copiar un atributo muy consultado (nombre_cliente en Pedidos) para evitar joins.
- **Atributos derivados / contadores:** guardar valores calculados (total factura, like_count, item_count). Se mantienen con triggers o columnas calculadas.
- **Tablas agregadas / aplanadas:** precalcular resumenes (ventas diarias).
- **Vista materializada:** tabla fisica que almacena el resultado de una consulta costosa. Se puede indexar. Hay que refrescarla (completa, incremental o programada). Oracle tiene Query Rewrite; PostgreSQL permite refresco concurrente.

12.3. Antes de desnormalizar (alternativas)

Indices (compuestos / de cobertura), ajuste de consultas (evitar SELECT *, paginar), cache (Redis), replicas de lectura.

12.4. Costes

- Amplificacion de escritura (mantener sincronizadas las copias).
- Riesgo de inconsistencia si falla la sincronizacion.
- Mayor uso de almacenamiento.

BLOQUE 13. EJERCICIOS PRACTICOS TIPO EXAMEN

Resuelve primero por tu cuenta y luego comprueba con las soluciones del Bloque 14.

PARTE A. Identificar dependencias funcionales a partir de un enunciado

Ejercicio A1. Una empresa guarda datos de empleados: cada empleado tiene un DNI unico, un nombre, y pertenece a un unico departamento. Cada departamento tiene un codigo unico, un nombre y un unico jefe (que es un empleado). Escribe las dependencias funcionales que se deducen del enunciado sobre el esquema EMPLEADO(DNI, Nombre, CodDpto, NombreDpto, DNI_Jefe).

Ejercicio A2. En una clinica: cada cita tiene un identificador unico; cada cita corresponde a un paciente (con su numero de historia y nombre) y a un medico (con su numero de colegiado y especialidad); para una cita se registran fecha y hora. Sobre CITA(IdCita, NumHistoria, NombrePaciente, NumColegiado, Especialidad, Fecha, Hora), indica las DF.

Ejercicio A3. En un instituto, un alumno se matricula en varias asignaturas y en cada asignatura obtiene una nota. Cada asignatura la imparte un unico profesor. Sobre MATRICULA(IdAlumno, NombreAlumno, IdAsignatura, NombreAsignatura, Nota, IdProfesor), escribe las DF y di cuales son parciales y cuales transitivas.

PARTE B. Calcular el cierre de atributos y hallar claves

Ejercicio B1. R(A, B, C, D, E) con F = { A -> B, B -> C, C -> D, D -> E }. Calcula A+ y di si A es clave candidata.

Ejercicio B2. R(A, B, C, D) con F = { AB -> C, C -> D, D -> A }. Calcula (AB)+, (BC)+ y (BD)+. Indica todas las claves candidatas y clasifica los atributos en primos / no primos.

Ejercicio B3. R(A, B, C, D, E, G) con F = { A -> BC, CD -> E, B -> D, E -> A }. Calcula (A)+ y (E)+. Es {A} superclave? Es clave candidata?

PARTE C. Normalizar (2FN, 3FN, BCNF)

Ejercicio C1. Dada VENTA(NumFactura, Fecha, CodCliente, NombreCliente, CodProducto, NombreProducto, Cantidad), con las DF:
- NumFactura -> Fecha, CodCliente
- CodCliente -> NombreCliente
- CodProducto -> NombreProducto
- NumFactura, CodProducto -> Cantidad

Indica la clave, en que forma normal esta y normaliza hasta 3FN.

Ejercicio C2. Dada R(A, B, C, D) con F = { AB -> C, C -> D, D -> B }. Determina las claves, los atributos primos y di en que forma normal esta (2FN/3FN/BCNF). Justifica.

Ejercicio C3. Dada INSCRIPCION(IdEstudiante, IdCurso, NombreCurso, NombreProfesor, Aula), con:
- IdEstudiante, IdCurso -> nada propio adicional (solo registra la inscripcion)
- IdCurso -> NombreCurso, NombreProfesor, Aula

Halla la clave, di la forma normal y normaliza a 3FN.

Ejercicio C4 (BCNF). R(Estudiante, Asignatura, Profesor) con las reglas: cada profesor imparte una sola asignatura; en una asignatura un estudiante tiene un solo profesor. DF:
- Estudiante, Asignatura -> Profesor
- Profesor -> Asignatura

Halla las claves, di si esta en 3FN y si esta en BCNF. Descompone a BCNF si hace falta.

PARTE D. Diseno conceptual (E-R / paso a tablas)

Ejercicio D1. Modela: una BIBLIOTECA tiene LIBROS; de cada libro hay varios EJEMPLARES (un ejemplar no existe sin su libro). Un SOCIO puede tomar prestados varios ejemplares y un ejemplar puede ser prestado por varios socios en el tiempo, guardando la fecha de prestamo. Identifica entidades fuerte/debil, cardinalidades y da el paso a tablas.

Ejercicio D2. En una universidad, una PERSONA puede ser ALUMNO y/o PROFESOR a la vez. Indica el tipo de jerarquia (disyuncion y completitud) y como se pasaria a tablas.

Ejercicio D3. Hay que registrar que un SUPERVISOR evalua el trabajo de un EMPLEADO en un PROYECTO concreto (la relacion EMPLEADO-PROYECTO recibe una evaluacion del supervisor). Que tecnica del MERE es estrictamente necesaria y por que?

BLOQUE 14. SOLUCIONES

Soluciones PARTE A

A1.
- DNI -> Nombre, CodDpto (el DNI determina al empleado y su departamento)
- CodDpto -> NombreDpto, DNI_Jefe (el codigo de departamento determina su nombre y su jefe)
- Por transitividad: DNI -> NombreDpto, DNI_Jefe.

Clave candidata: {DNI}. Hay dependencia transitiva (NombreDpto y DNI_Jefe dependen de CodDpto, que no es clave): NO esta en 3FN.

A2.
- IdCita -> NumHistoria, NumColegiado, Fecha, Hora
- NumHistoria -> NombrePaciente
- NumColegiado -> Especialidad
- Transitivas: IdCita -> NombrePaciente (via NumHistoria) e IdCita -> Especialidad (via NumColegiado).

Clave: {IdCita}. Hay dependencias transitivas -> no esta en 3FN.

A3.
- IdAlumno -> NombreAlumno
- IdAsignatura -> NombreAsignatura, IdProfesor
- IdAlumno, IdAsignatura -> Nota

Clave candidata: {IdAlumno, IdAsignatura}.
- Parciales: NombreAlumno depende solo de IdAlumno (parte de la clave); NombreAsignatura e IdProfesor dependen solo de IdAsignatura (parte de la clave). => viola 2FN.
- Transitiva: si IdAsignatura -> IdProfesor y un profesor tuviera nombre, ese nombre seria transitivo. Aqui la transitividad aparece tras corregir las parciales.

Soluciones PARTE B

B1. A+ : empiezo {A}. A->B da {A,B}; B->C da {A,B,C}; C->D da {A,B,C,D}; D->E da {A,B,C,D,E}. A+ = todos => {A} es superclave, y como es un solo atributo es minima => CLAVE CANDIDATA. Si.

B2. F = { AB->C, C->D, D->A }.
- (AB)+ = {A,B} -> AB->C da C -> {A,B,C} -> C->D da D -> {A,B,C,D} = todos. {AB} es clave candidata.
- (BC)+ = {B,C} -> C->D da D -> {B,C,D} -> D->A da A -> {A,B,C,D} = todos. {BC} es clave candidata.
- (BD)+ = {B,D} -> D->A da A -> {A,B,D} -> AB->C da C -> {A,B,C,D} = todos. {BD} es clave candidata.

Claves candidatas: {AB}, {BC}, {BD}. Atributos primos: A, B, C, D (todos aparecen en alguna clave). No primos: ninguno.

B3. F = { A->BC, CD->E, B->D, E->A }.
- (A)+ = {A} -> A->BC da {A,B,C} -> B->D da {A,B,C,D} -> CD->E da {A,B,C,D,E} -> E->A nada nuevo. (A)+ = {A,B,C,D,E}. Falta G. {A} NO es superclave (no incluye G).
- (E)+ = {E} -> E->A da {A,E} -> A->BC da {A,B,C,E} -> B->D da {A,B,C,D,E} -> CD->E nada nuevo. (E)+ = {A,B,C,D,E}. Tampoco incluye G.

G no aparece en el lado derecho de ninguna DF, asi que debe estar en TODA clave. Una clave candidata seria {A,G} (porque A+ = todo menos G, anadiendo G se completa). {A} no es clave; {A,G} si.

Soluciones PARTE C

C1. Clave candidata: {NumFactura, CodProducto} (su cierre da todo).
- 1FN: si.
- 2FN: NO. Fecha y CodCliente dependen solo de NumFactura (parte de la clave); NombreProducto depende solo de CodProducto (parte de la clave) -> dependencias parciales.

Normalizacion a 3FN:
- FACTURA(NumFactura PK, Fecha, CodCliente FK)
- CLIENTE(CodCliente PK, NombreCliente) -- separa la transitiva

NumFactura->CodCliente->NombreCliente
- PRODUCTO(CodProducto PK, NombreProducto)
- DETALLE(NumFactura FK, CodProducto FK, Cantidad) con PK (NumFactura, CodProducto)

C2. R(A,B,C,D), F = { AB->C, C->D, D->B }.
Claves candidatas: {AB}, {AC}, {AD} (calculando cierres, A siempre necesario; ver B2 razonamiento analogo). Atributos primos: A, B, C, D (todos). No primos: ninguno.
- Como TODOS los atributos son primos, esta automaticamente en 3FN (y en 2FN).
- BCNF: NO. En C->D y D->B, ni C ni D son superclaves (sus cierres no contienen todos los atributos), pero determinan atributos. => viola BCNF.

Respuesta: esta en 3FN pero NO en BCNF.

C3. Clave candidata: {IdEstudiante, IdCurso}.
- 2FN: NO. NombreCurso, NombreProfesor y Aula dependen solo de IdCurso (parte de la clave) -> parcial.

Normalizacion a 3FN:
- CURSO(IdCurso PK, NombreCurso, NombreProfesor, Aula)
- INSCRIPCION(IdEstudiante FK, IdCurso FK) con PK (IdEstudiante, IdCurso)

(Nota: si NombreProfesor dependiera de un IdProfesor, se separaria tambien PROFESOR.)

C4. R(Estudiante, Asignatura, Profesor), F = { EA->P, P->A }.
- Claves candidatas: {Estudiante, Asignatura} y {Estudiante, Profesor}. (EA+ = todo; EP+ : P->A da A, luego EA->P ya tengo P, => {E,P,A} = todo.)
- Atributos primos: Estudiante, Asignatura, Profesor (todos). => esta en 3FN (no hay no primos).
- BCNF: NO. La DF P->A tiene determinante P, que NO es superclave (P+ = {P,A}, no incluye Estudiante). => viola BCNF.

Descomposicion a BCNF:
- R1(Profesor PK, Asignatura) -- de P->A
- R2(Estudiante, Profesor) con PK (Estudiante, Profesor)

Esta descomposicion conserva los datos, aunque no preserva la DF EA->P (caso clasico: BCNF puede no preservar todas las dependencias).

Soluciones PARTE D

D1.
- LIBRO: entidad fuerte (PK: ISBN). EJEMPLAR: entidad debil (depende del libro). Relacion LIBRO 1:N EJEMPLAR (identificadora), EJEMPLAR participacion total.
- SOCIO: entidad fuerte (PK: NumSocio). PRESTAMO: relacion N:M entre SOCIO y EJEMPLAR con atributo FechaPrestamo.

Paso a tablas:
- LIBRO(ISBN PK, Titulo, ...)
- EJEMPLAR(ISBN FK, NumEjemplar) con PK (ISBN, NumEjemplar) -- PK compuesta = clave del dueno + discriminante
- SOCIO(NumSocio PK, Nombre, ...)
- PRESTAMO(NumSocio FK, ISBN, NumEjemplar, FechaPrestamo) con PK que incluye las claves + la fecha si un mismo ejemplar puede prestarse al mismo socio varias veces.

D2. Jerarquia de PERSONA con subclases ALUMNO y PROFESOR.
- Disyuncion: SOLAPADA (una persona puede ser ambas a la vez).
- Completitud: depende del enunciado; si toda persona debe ser alumno o profesor, es TOTAL; si puede ser otra cosa, PARCIAL.

Paso a tablas (opcion una tabla por clase):
- PERSONA(DNI PK, Nombre, ...)
- ALUMNO(DNI PK/FK, NotaMedia, ...)
- PROFESOR(DNI PK/FK, Departamento, ...)

Al ser solapada, un mismo DNI puede aparecer a la vez en ALUMNO y en PROFESOR.

D3. Es estrictamente necesaria la AGREGACION. Motivo: la evaluacion del SUPERVISOR no recae sobre EMPLEADO ni sobre PROYECTO por separado, sino sobre la RELACION entre ambos (TRABAJA_EN). Como una relacion no puede vincularse directamente con otra relacion ni recibir un nuevo vinculo de una tercera entidad, se agrega TRABAJA_EN (junto con EMPLEADO y PROYECTO) tratandola como una entidad compuesta, y se conecta esa agregacion con SUPERVISOR mediante la relacion EVALUA.

ANEXO. CHULETA RAPIDA (ultimo repaso)

- DF X -> Y: mismas X => mismas Y. Unidireccional.
- Cierre X+: arranca en X; anade derechos de DF cuyo izquierdo ya esta dentro.
- X+ = todos => superclave; superclave minima => candidata.
- Primo = esta en alguna candidata. Todos primos => 3FN automatica.
- 1FN: valores atomicos.
- 2FN: 1FN + sin dependencias parciales (solo peligra con clave compuesta).
- 3FN: 2FN + sin dependencias transitivas.
- BCNF: todo determinante es superclave.
- Parcial = no primo depende de PARTE de la clave. Transitiva = no primo depende de otro no primo.
- 1:N -> FK en el lado N. N:M -> tabla intermedia con PK compuesta. Multivaluado -> tabla aparte.
- Entidad debil -> PK = clave del dueno + discriminante.
- Agregacion -> relacion que participa en otro vinculo con una tercera entidad.
- Jerarquia: disjunta/solapada (cuantas subclases) + total/parcial (obligatorio o no).
- Integridad referencial: FK CASCADE / SET NULL / RESTRICT.
- Desnormalizar = anadir redundancia a proposito para acelerar lecturas (ultimo recurso).